

Evolutionary Robotics

2: Evolutionary Algorithms

Prof. Dr. Javad Ghofrani



**Hochschule
Bonn-Rhein-Sieg**
University of Applied Sciences

A young man with brown hair, wearing a maroon sweater and a black backpack, is looking into an open grey safe. He is holding a yellow sticky note. The safe has a digital keypad and a handle. The background is a plain, light-colored wall.

check your notes from last
week

Outline

Introduction
Why Evolutionary Robotics?

Biological fundamentals
Natural Selection, neural Systems

Evolutionary algorithms
Selection, Mutation, Crossover

Artificial neural networks
Controller for robots

How to evolve robots
(Encoding, Fitness, Simulation)

Evolution of simple navigation
(Phototaxis, Obstacle avoidance)

Power and Limitations of Reactive
Intelligence

Evolving Morphology
(Body + Brain Co-evolution)

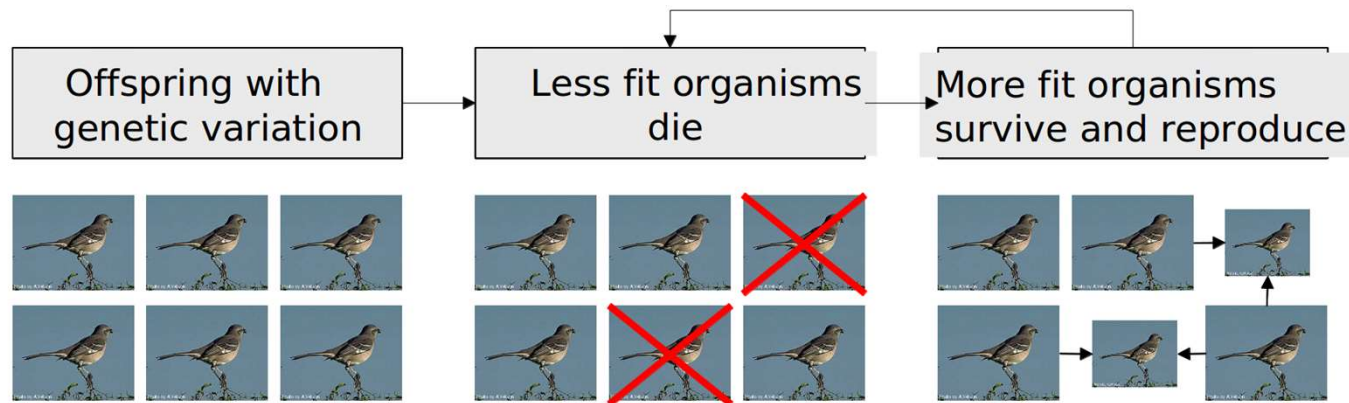
Challenges and new approaches
Reality Gap, Diversity, DNNs

Learning Objectives

By the end of this session, students will be able to:

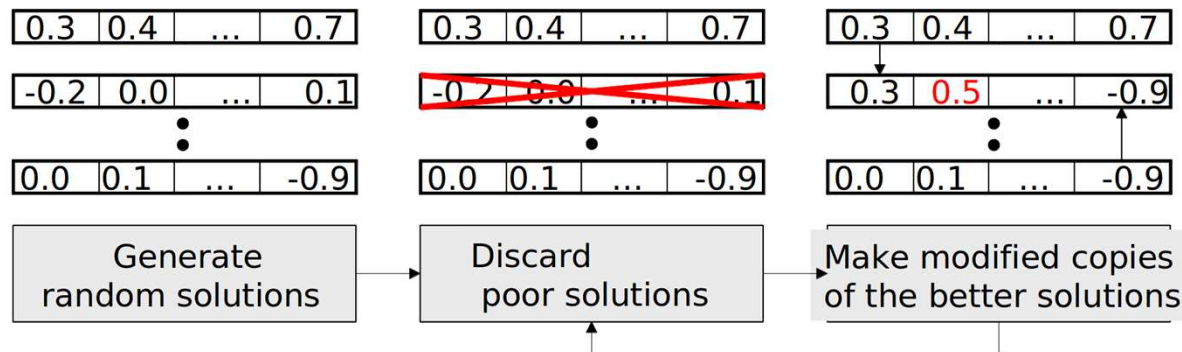
- Explain the fundamental principles of Genetic Algorithms (GAs) and describe their similarities and differences to natural evolution.
- Apply Genetic Algorithms to formulate and solve optimization problems.
- Identify and compare selection methods used in artificial evolution through Genetic Algorithms.

“Evolutionary Algorithms in 3 minutes”



BIOLOGICAL EVOLUTION

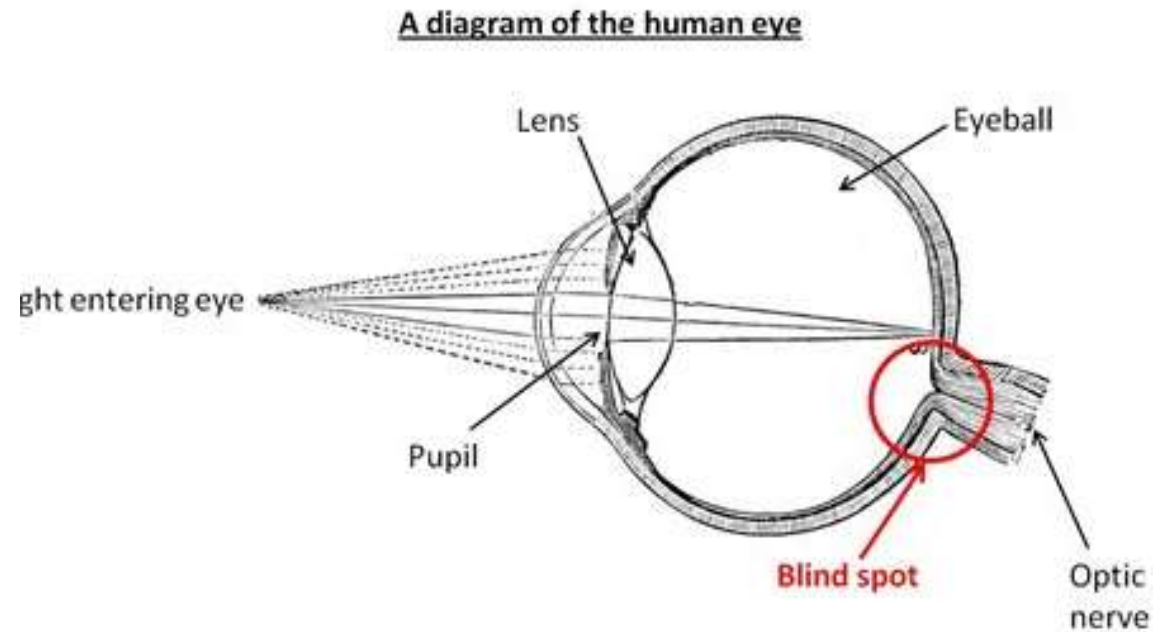
THE GENETIC ALGORITHM



Natural selection vs. optimization

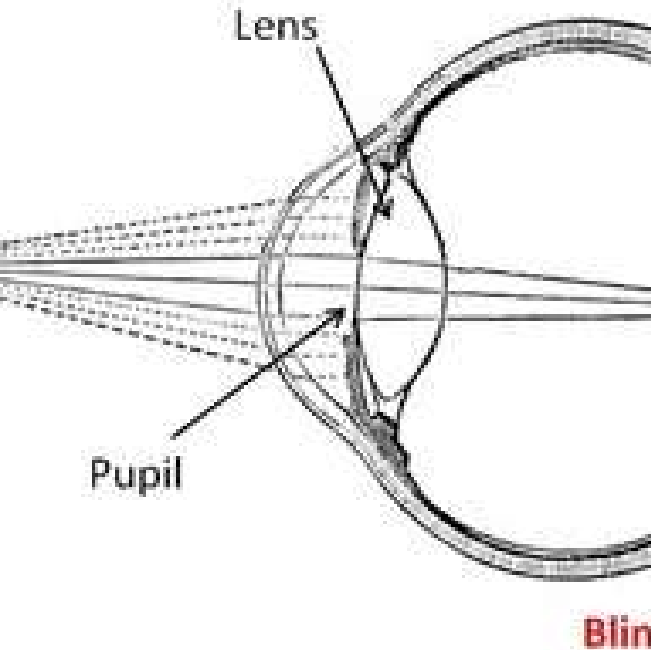
Evolutionary algorithms are methods of **optimization**, but natural selection is **not identical to an optimizer**; survival is not directly bound to optimality.

natural evolution has no foresight and sticks to 'good enough' solutions (cf. local optima, example: blind spot in visual field of eyes of vertebrates)



Natural selection vs. optimization

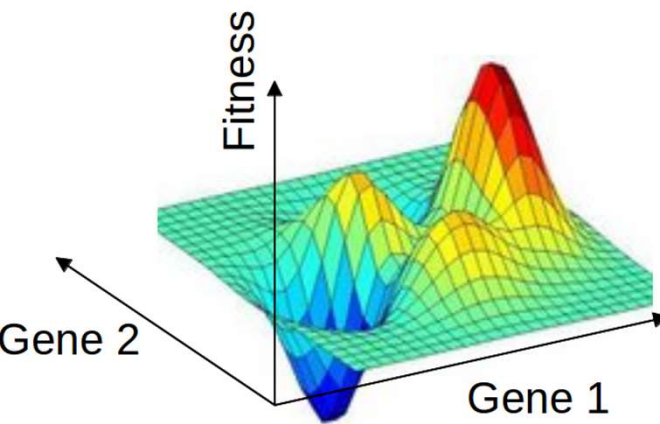
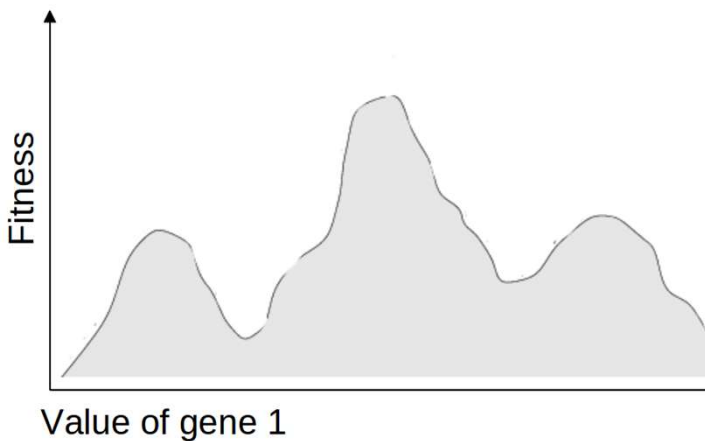
A diagram of the human eye



not perfectly adapted, only fit enough:

Two hikers are surprised in the woods by a Grizzly Bear. One prepares to run. The other says “Don’t be silly. You can’t outrun a Grizzly.” The first replies “I don’t need to outrun a Grizzly, I only need to outrun you.”

Explanatory model: fitness landscapes



[disclaimer: reality doesn't look that nice!]

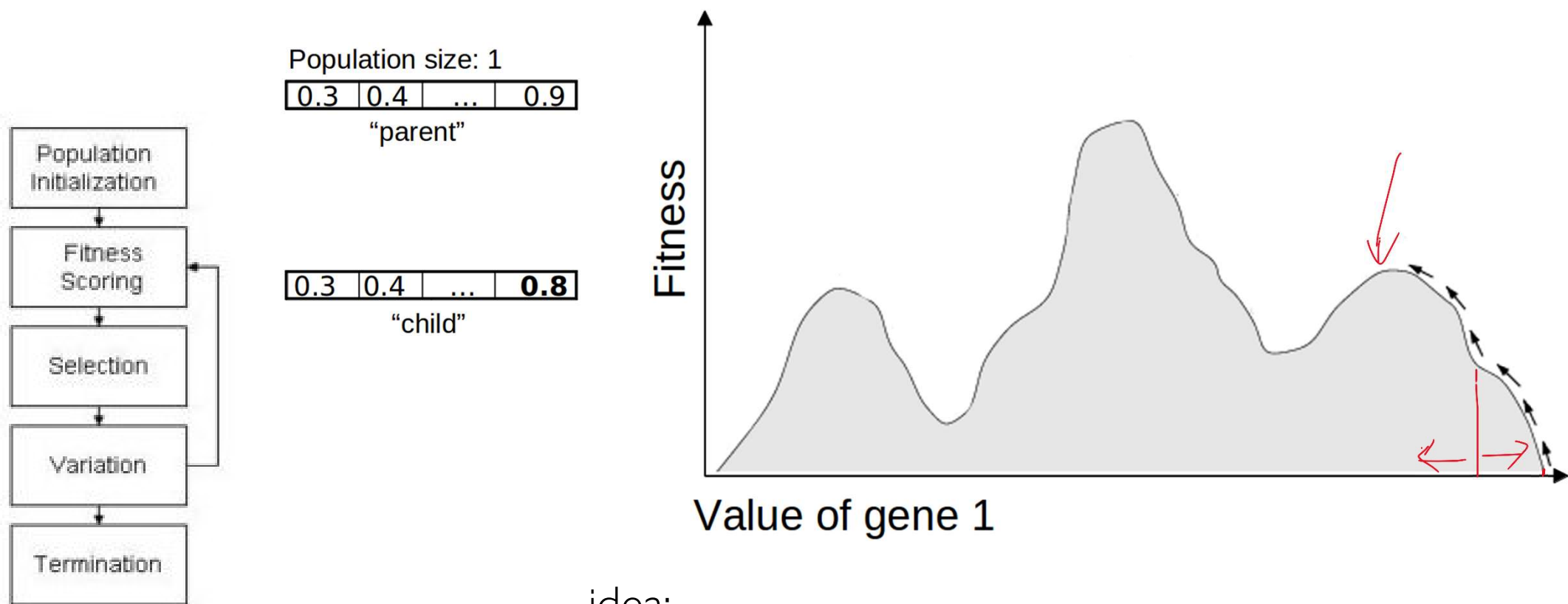
If a **genotype** has n genes, the fitness landscape exists in $n + 1$ dimensions: the fitness of the phenotype produced by that genotype is the extra dimension.

genotype: genetic makeup of an organism (collection of genes)

phenotype: observable characteristic of an organism (morphology, behavior etc.)

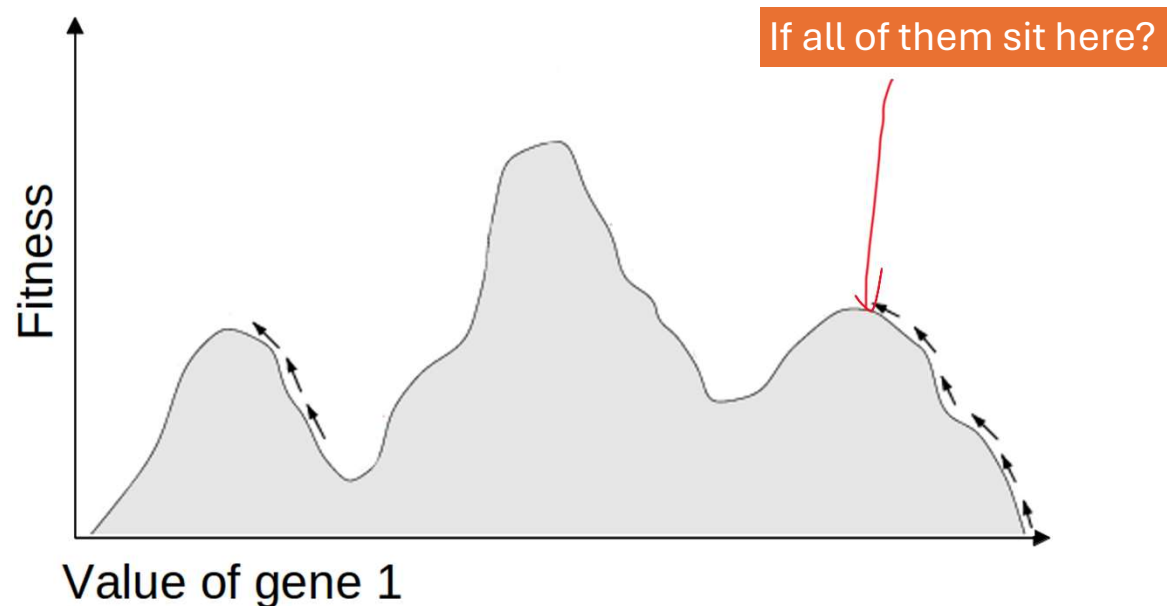
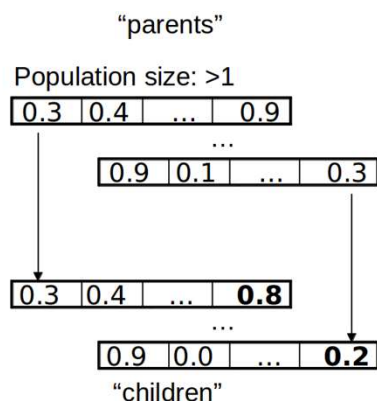
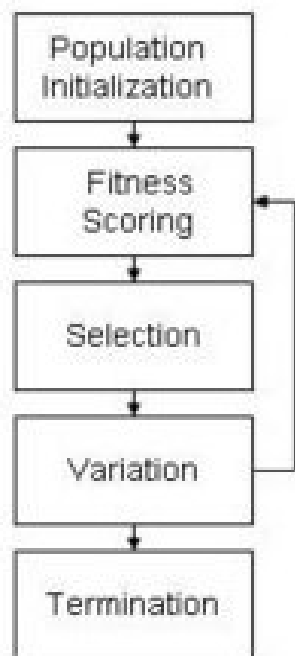
gene: smallest unit of genetic information

Optimization: The hill climber



idea:
small changes towards bigger fitness

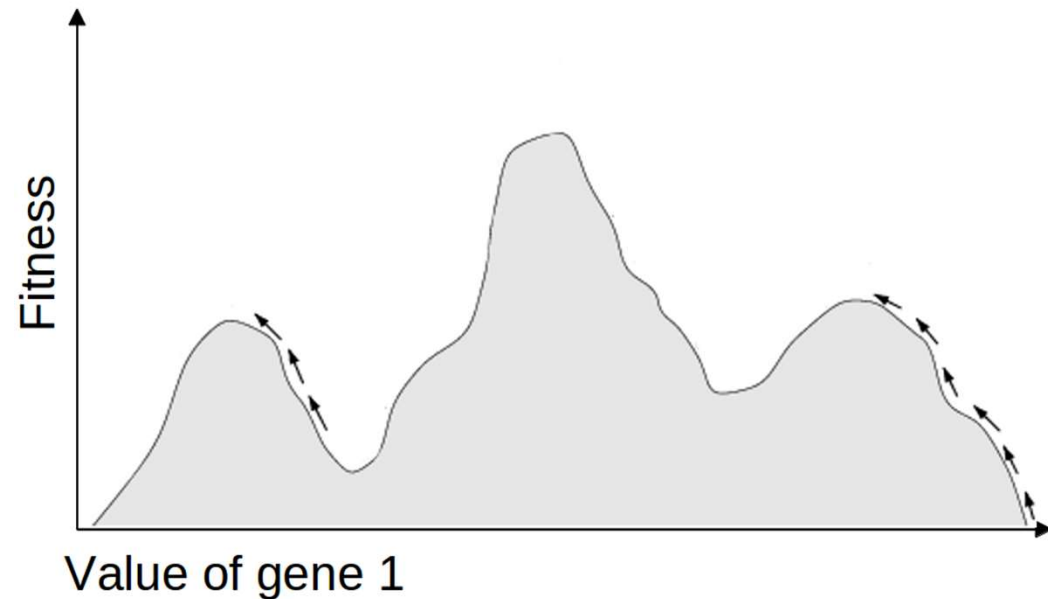
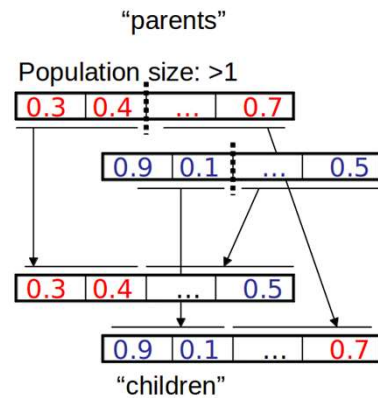
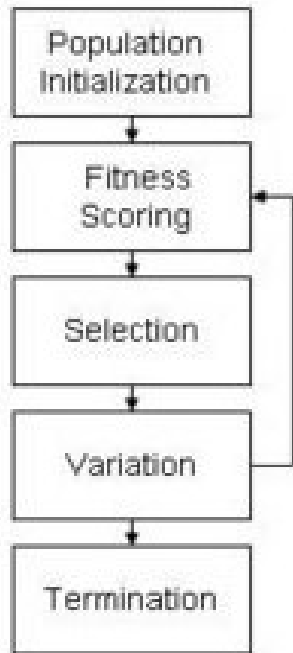
Optimization: The parallel hill climber



idea:
small changes towards bigger fitness **in parallel**
(population),

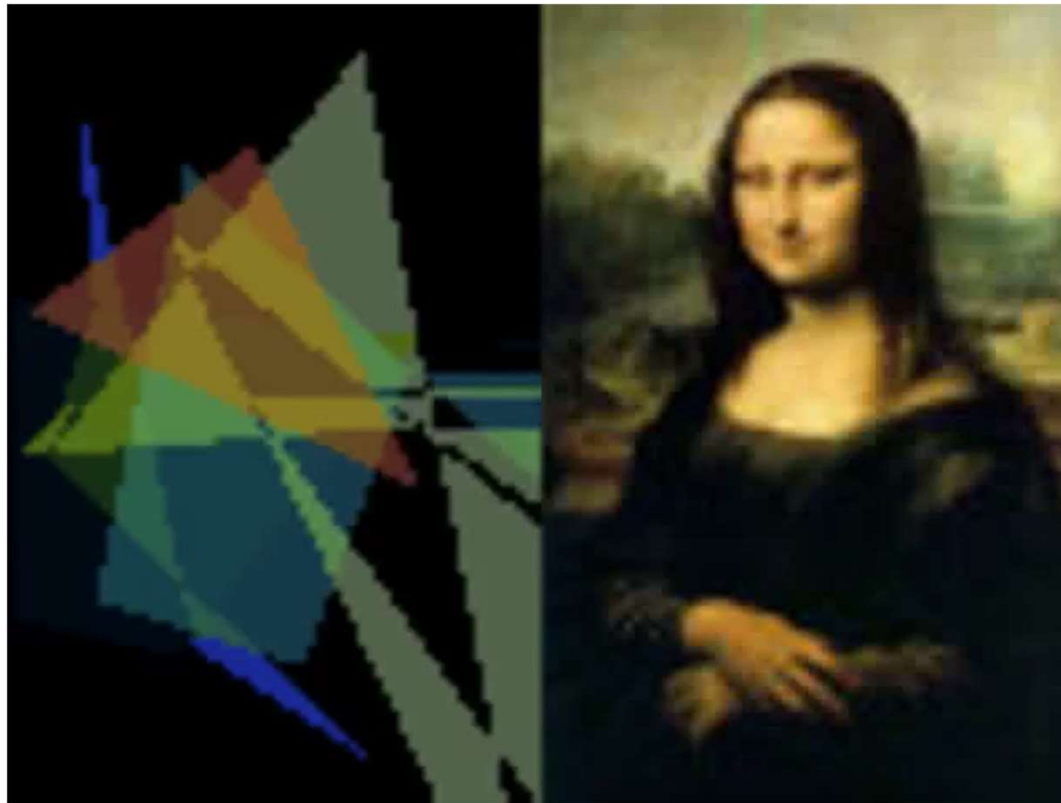
starting at multiple positions

Optimization: The Genetic Algorithm



idea:
population
+ small changes (mutation)
+ recombination (e.g., 1-point crossover)
+ selection

Example: “evolving Mona Lisa”



<http://www.youtube.com/watch?v=S1ZPSbImvFE>

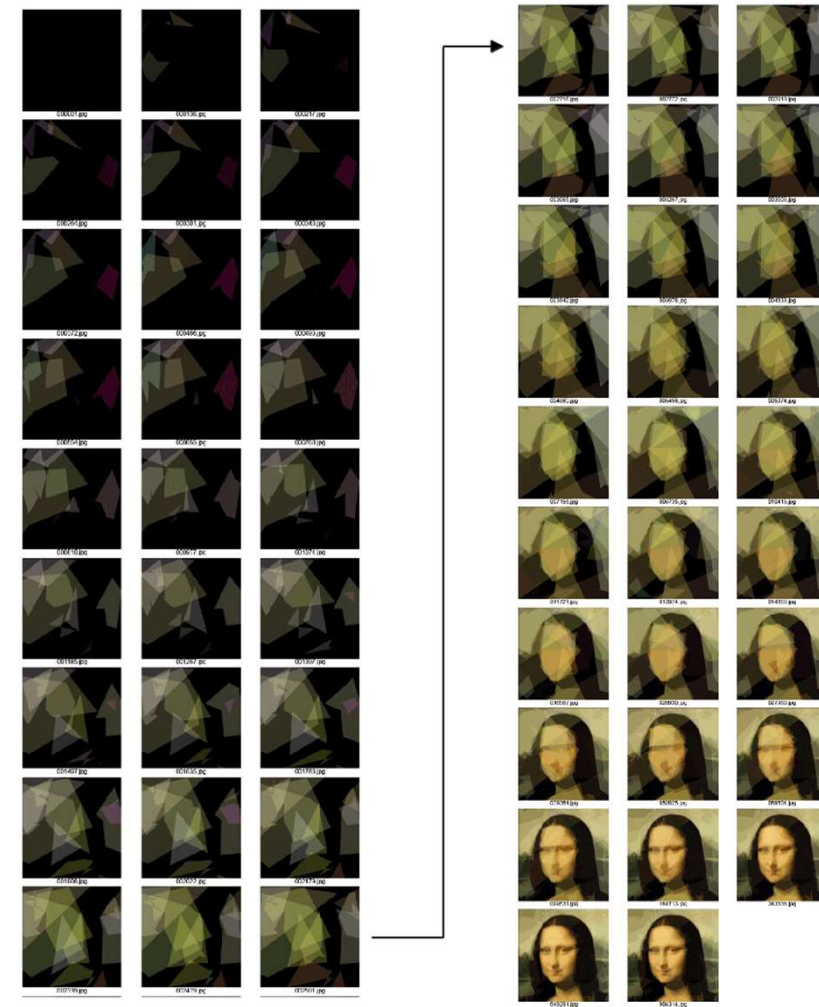
Example: “evolving Mona Lisa”

What is the phenotype?

How to compute fitness?

What is a good genotype?

Any idea how the fitness landscape might look like?



Artificial evolution

The purpose of artificial evolution is the automatic generation of solutions to hard problems.

there are three fundamental components:

population of potential solutions: implements a parallel search

variation operators (recombination and mutation): generate diversity and renewal

selection: pressure towards higher fitness

(note, the '4 pillars of evolution' include diversity & heredity instead of variation operators)

Artificial and natural: similarities

Similarities between natural and artificial evolution:

- general concept of phenotype (e.g., computer program, object shape, electronic circuit, behavior, robot design, etc.)
- general concept of genotype (genetic representation of the phenotype)
- population
- diversity
- selection
- heredity

Artificial and natural: differences

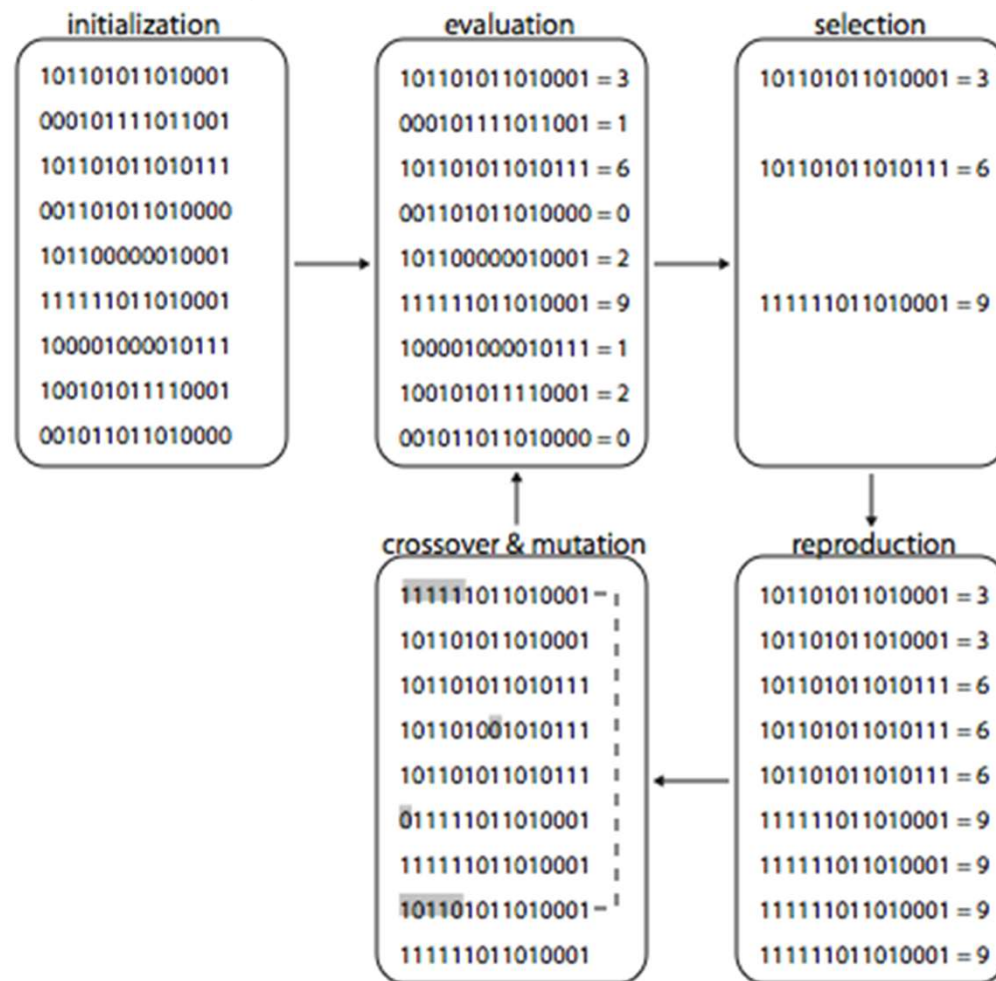
Differences of artificial evolution compared to natural evolution:

- fitness is a **direct measure of performance** of a solution to the problem
- **direct selection** of the best according to performance criterion (fitness function)
- **expected improvement** between initial and final solution (progress of natural evolution is not necessarily an improvement)
- evolutionary runs operate on a clearly defined **time segment** (natural evolution has no obvious start and virtually no end)
- no RNA, no transcription/translation, no protein folding optimization (vs. 'fit enough')

(see also slide 'Genetic representation')

Recipe for Artificial Evolution

Recipe for the general Evolutionary Algorithm



Recipe for the general Evolutionary Algorithm

- devise genetic representation
- design a fitness function
- choose selection method
- choose crossover & mutation
- build a population
- choose data analysis method (interpretation of the results might be nontrivial)

Repeat generation cycle until:

- Maximum fitness value is found (if defined)
- Solution found is good enough
- No fitness improvement for several generations

Genetic representation (1/2)

Evolutionary algorithms are **applicable to any problem domain** as long as suitable genetic representation, fitness, and genetic operators are/can be chosen.

genetic representation describes elements of genotype and mapping to phenotype

- must match genetic operators of recombination
 - Mutation set of possible genotypes should include optimal solution to the problem
- ⇒ completeness criterion

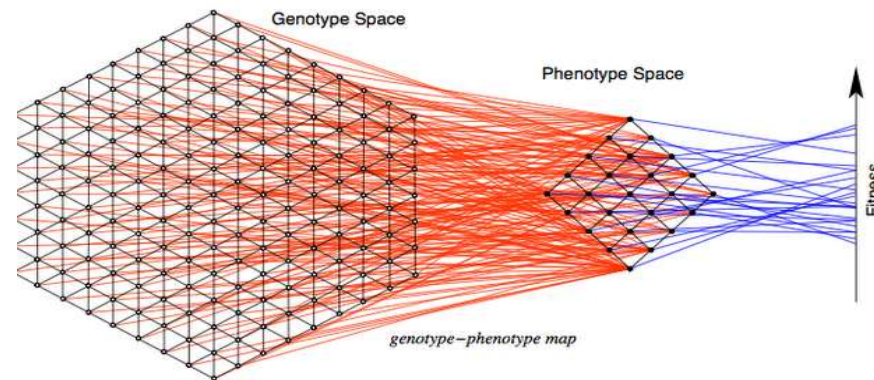
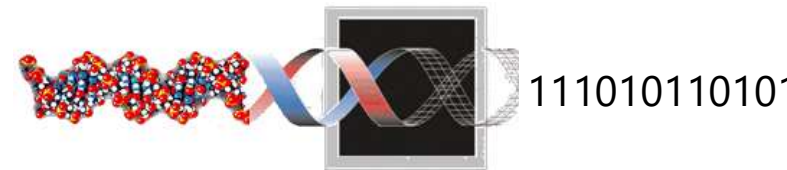


Figure 1. Fitness and Genotype-Phenotype map. Redrawn after [20]

Genetic representation (2/2)

Choice of representation benefits from domain knowledge:

- encoding of relevant parameters
- appropriate resolution of parameters



Great simplification of genetics:

- single stranded sequence of characters (e.g., binary)
- fixed length (natural genomes have variable length)
- often haploid structure and one chromosome
- often one-to-one direct correspondence between gene and parameter
- gene expression and genetic regulation used only in specific situations (e.g., we do not turn on/off the genes)

Types of representations (1/2)

The 3 most common types are:

discrete, real-valued, and tree-based

discrete: a sequence of L discrete values drawn from an alphabet of cardinality k , e.g., binary string ($|\{0,1\}| = 2 = k$) of $L = 8$ positions: **01010100**

other examples: integers, permutations, job schedules

real-valued: genotype is sequence of real values that represent parameters, used when high-precision parameter optimization is required (implemented as floating point)

Types of representations (2/2)

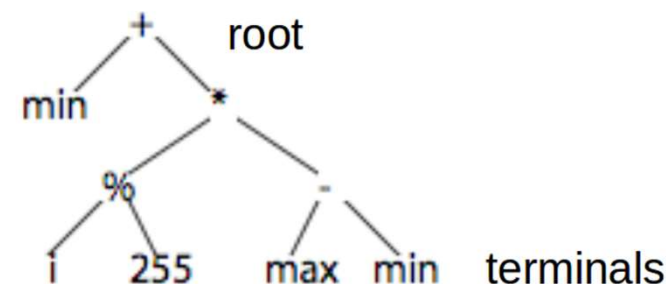
Tree-based: genotype describes a tree with branching points and terminals, suitable for encoding hierarchical structures, e.g., used to encode computer programs

a computer program is made of:

- operators (function set: multiplication, if-then, log, etc.)
- operands (terminal set: constants, variables, sensor readings, etc.)

Expression $r = \min + (i/255)(\max - \min)$

Nested List $(+, \min, (*, (/i, 255), (-, \max, \min)))$



How to initialize the population?

Size: sufficiently large to cover problem space (!), but sufficiently small for evaluation costs (typical size: between 10s and 1000s of individuals)

Uniform sample of search space + ensure **correctness**:

Binary strings: **0** or **1** with probability **0.5**

Real-valued representations: uniform on a given interval if bounded phenotype (e.g., $[-2.0, +2.0]$)

- otherwise, best guess or other methods

Trees are built recursively starting from root:

- root is randomly chosen from function set;
- for every branch, randomly choose among all elements of function set and of terminal set;
- if terminal is chosen, it becomes leaf;
- set maximum depth of tree

Fitness function

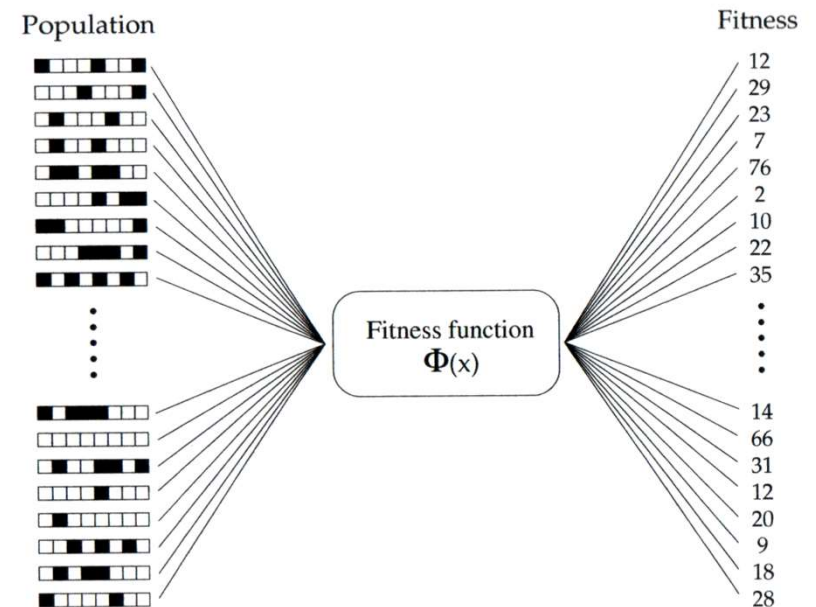
Evaluates performance of phenotype with a numerical score

can be a combination of components (average speed minus number of collisions etc.)

depending on the application,
sometimes difficult to define properly

(will be discussed again later on)

(see also Nolfi & Floreano p. 20)



Example: an optimization problem

Traveling salesman problem

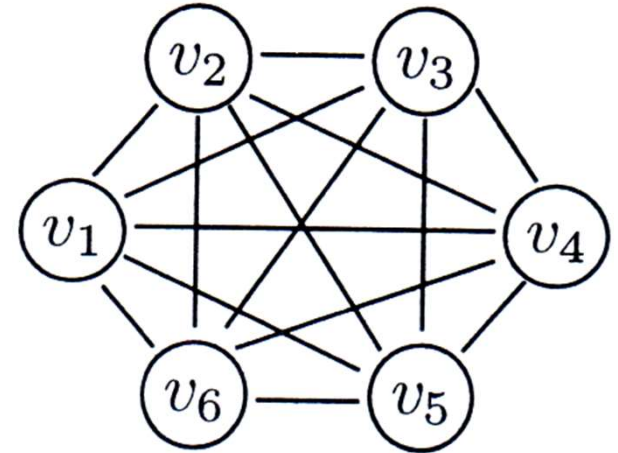
- NP-hard problem in combinatorial optimization
- undirected weighted graph
- here with just $n = 6$ vertices
- a complete graph has $\frac{n(n-1)}{2}$ edges

here:

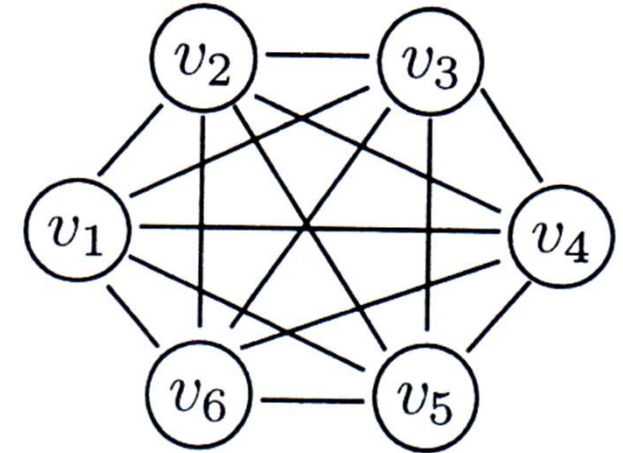
15 edges

fitness is length of the route

Q: How many potential solutions are there?



Example: an optimization problem



Q: How many potential solutions are there?

a permutation over the vertices uses 6 of these edges

there are $6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$ permutations

but packages of **12** permutations each are identical in terms of graph theory (each of the **6** vertices can be the start and there are two directions of travel),

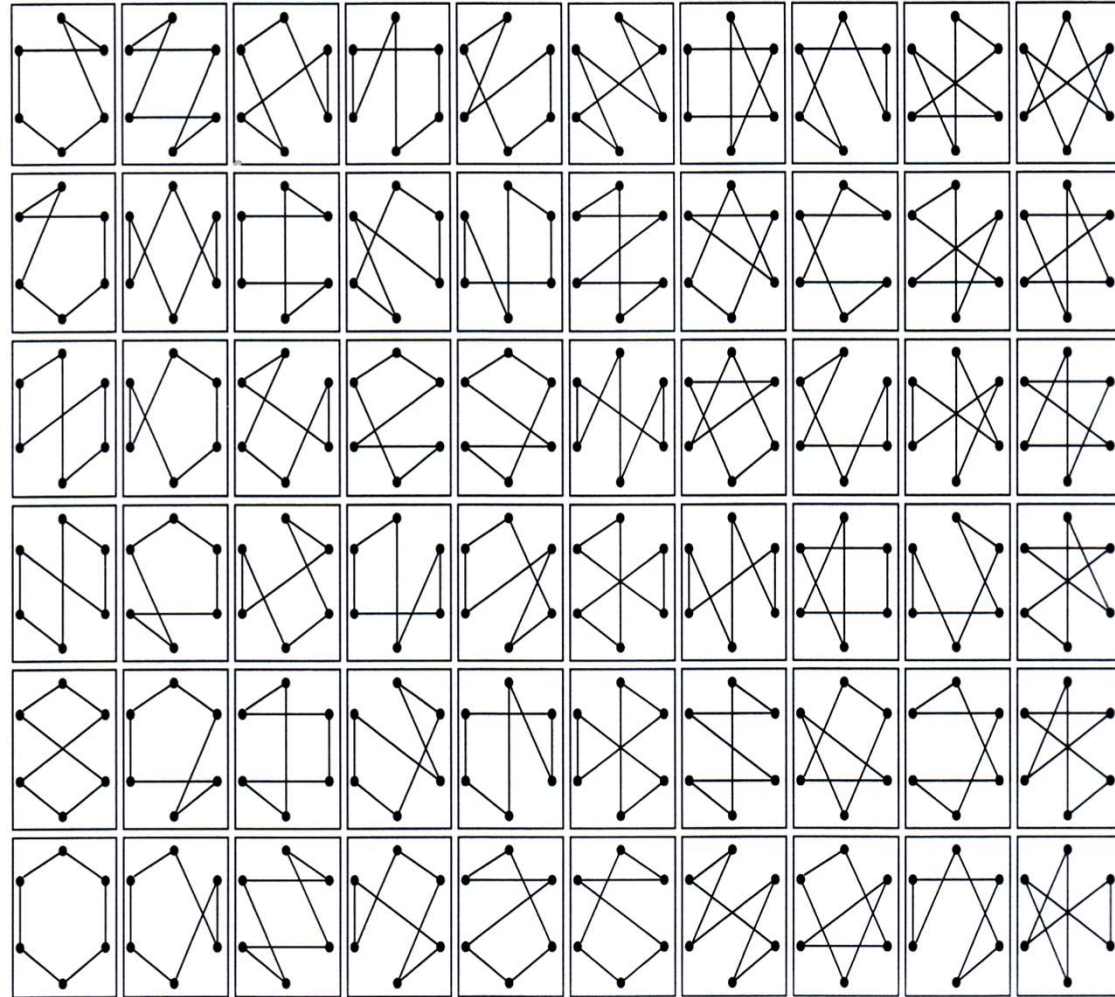
hence we get $720/12 = 60$

Example: Search space (unsorted enumeration)

Fitness function?

Fitness landscape?

Mutation operator?



Example: Genome & neighborhood in search space

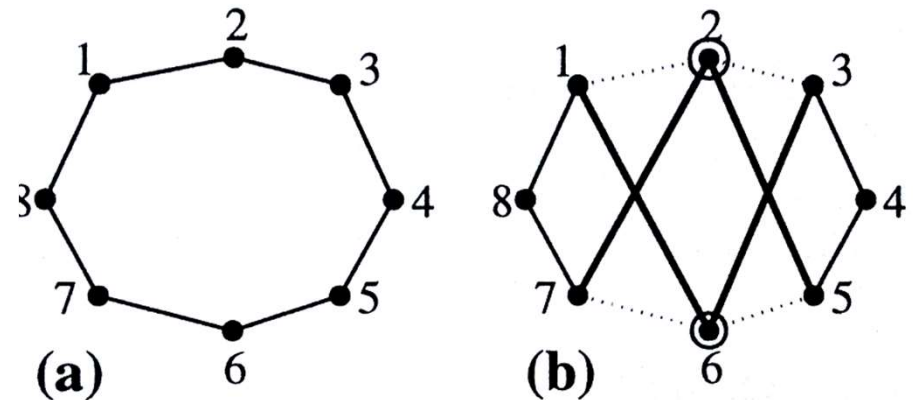
Genome is a permutation, e.g. (1,2,3,4,5,6,7,8) or (3,7,6,1,4,2,8,5) concept of incorrect genomes, e.g. (1,2,3,4,5,5,7,8)

Ideas for a mutation operator?

Example: Genome & neighborhood in search space

genome is a permutation, e.g. (1,2,3,4,5,6,7,8) or (3,7,6,1,4,2,8,5) concept of incorrect genomes, e.g. (1,2,3,4,5,5,7,8)

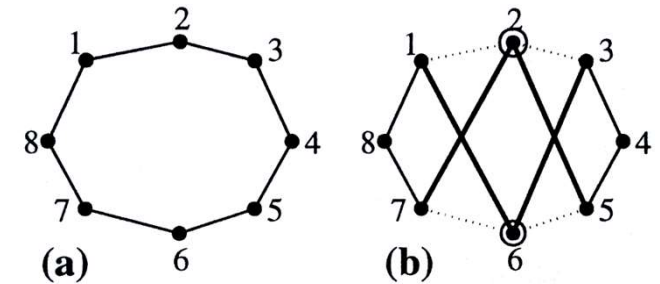
Ideas for a mutation operator?



swapping: pick two cities in the permutation (1,2,3,4,5,6,7,8) and swap them: (1,6,3,4,5,2,7,8)

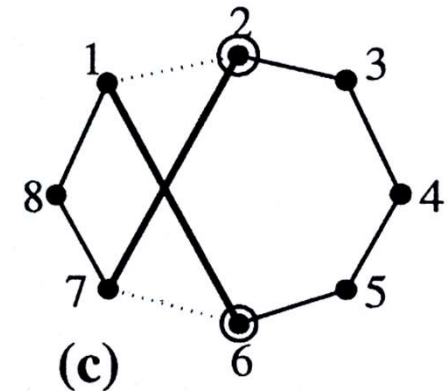
Example: Genome & neighborhood in search space

Genome is a permutation, e.g. (1,2,3,4,5,6,7,8) or (3,7,6,1,4,2,8,5) concept of incorrect genomes, e.g. (1,2,3,4,5,5,7,8)



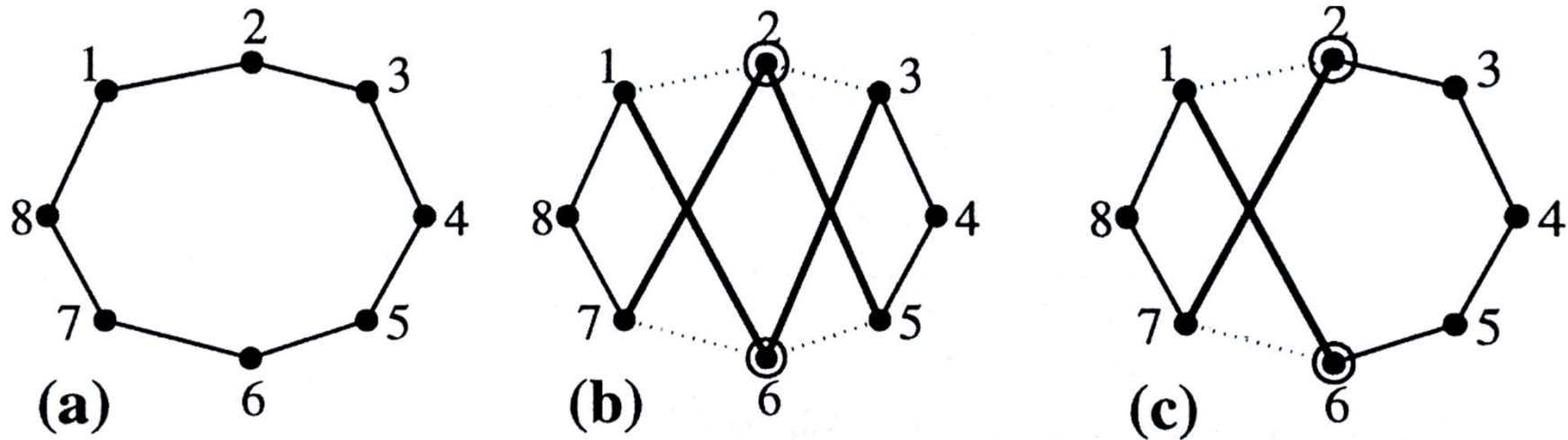
Ideas for a mutation operator?

invert: pick two cities in the permutation (1,2,3,4,5,6,7,8) and invert the sub-sequence: (1,6,5,4,3,2,7,8)



Example: Genome & neighborhood in search space

the latter changes only 2 edges
⇒ smaller change ⇒ fine-grained local search



Selection

A method to make sure that better individuals make comparatively **more offspring**
used in artificial evolution and breeding



selection pressure is inversely proportional to number of selected individuals

high selection pressure
⇒ rapid **loss of diversity** and early **convergence**
⇒ **tradeoff**: selection vs. diversity

make sure that also less performing individuals have a non-zero chance to reproduce to some extent to preserve a certain degree of diversity

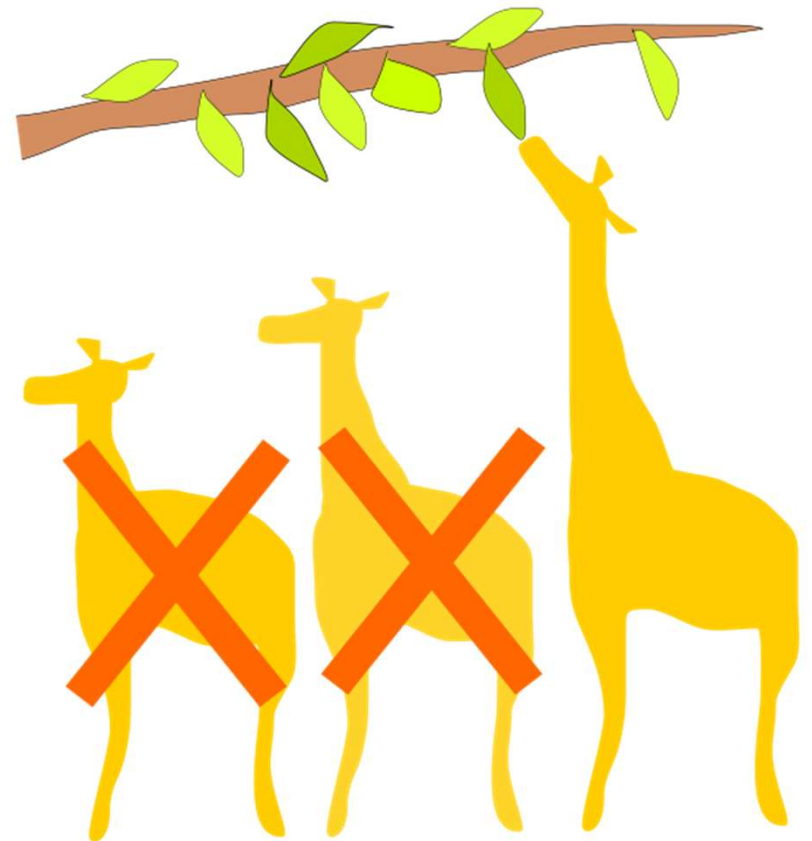
Selection methods

How to implement the selection to allow a non-zero chance for 'bad' individuals while picking 'good' ones more often?

There is a number of selection methods for evolutionary algorithms.

Examples are:

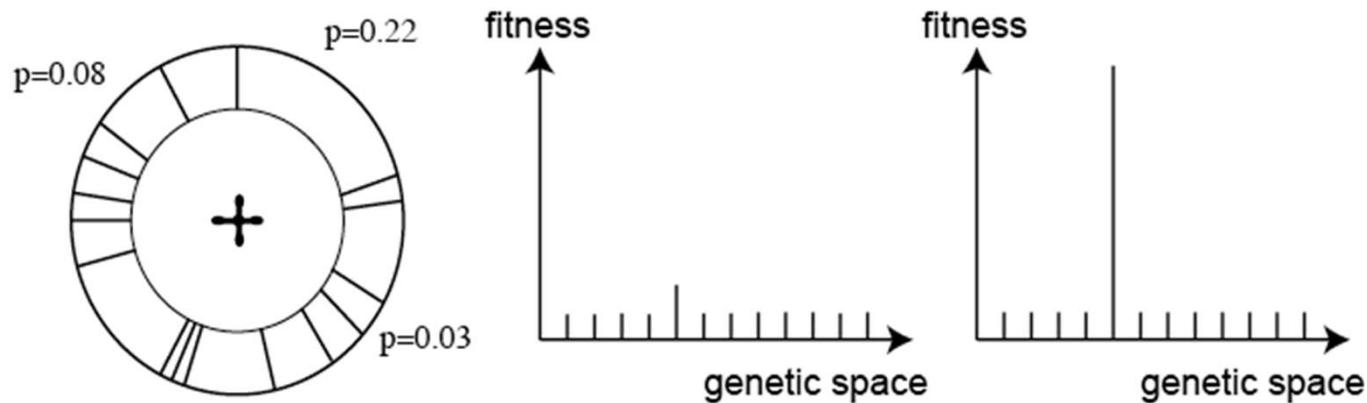
- proportionate selection
- rank-based selection
- tournament selection



Proportionate selection

The probability that an individual i makes an offspring is proportional to how good its fitness $f(i)$ is with respect to the population fitness:

$$p(i) = \frac{f(i)}{\sum_{j \in \{1, \dots, N\}} f(j)} \quad (1)$$



Proportionate selection

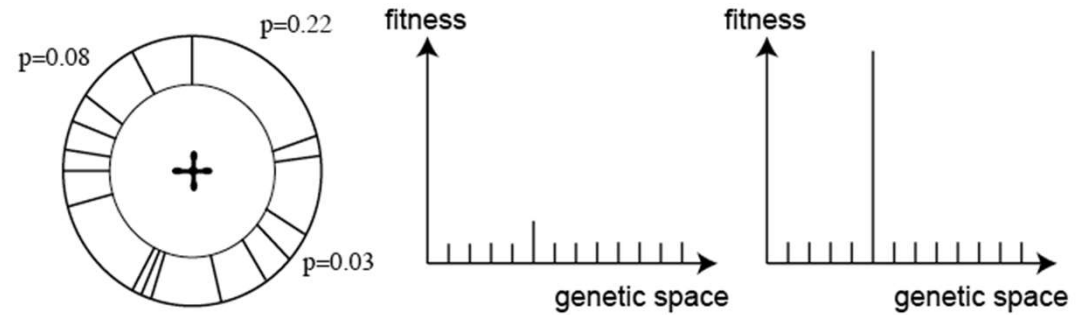
Problems:

almost uniform fitness values

⇒ random search

few high-fitness individuals

⇒ high selection pressure



(see also Nolfi & Floreano p. 21)

Rank-based selection

Individuals are sorted on their fitness value from best to worse.

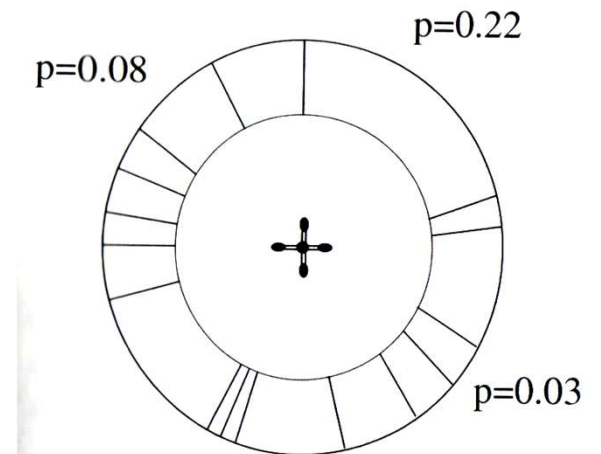
The place in this sorted list is called the rank r .

Instead of using the fitness value of an individual i , the rank $r(i)$ is used to select individuals:

$$p(i) = 1 - \frac{r(i)}{\sum_{j \in \{1, \dots, N\}} r(j)} \quad (2)$$

then use roulette wheel

individual	fitness	rank
A	5	5
B	7	3
C	8	2
→ D	2	8
E	3	7
→ F	9	1
G	7	4
H	4	6



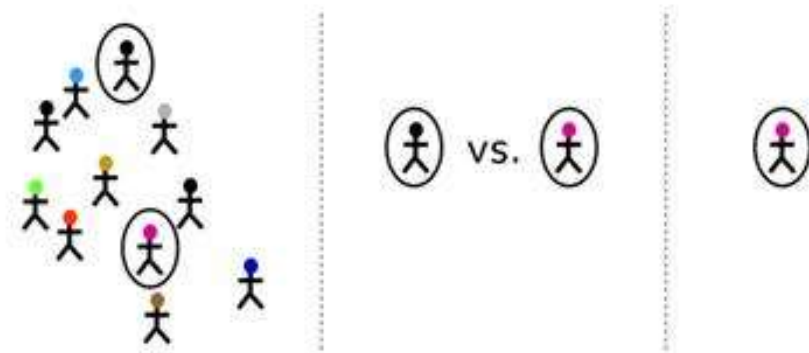
Tournament selection

Idea: randomly selecting a small group (tournament) of individuals from a population and choosing the individual with the best fitness score from that group to be a parent. repeated multiple times to select enough parents for the next generation.

- tournament size is k
- we pick a group of k individuals from the population randomly the best individual of this group is selected

Variant:

- the best individual is only selected with probability $P < 1$
- the 2nd best is selected with probability $P(1 - P)$
- 3rd best with $P((1 - P)^2)$
- etc.



Example

Candidate #	Candidate String	Matching Characters	Fitness
1	HELXO WORQD	H,E,L,O, ,W,O,R,_,D → 9 matches	9
2	HEALO WORLF	H,E,,L,O, ,W,O,R,,_ → 8 matches	8
3	HELLO WXRLD	all but 8th character	10
4	HXLLO WIRLD	8 matches	8
5	HELLO WOROD	all but 10th character	10
6	HELXO WQRLD	9 matches	9
7	HELLO WORLT	10 matches	10
8	HELPO WOTLD	8 matches	8
9	HELLO WORLE	10 matches	10
10	HEXLO WORZD	8 matches	8

Use one of the selection methods for parents of next generation

Replacement



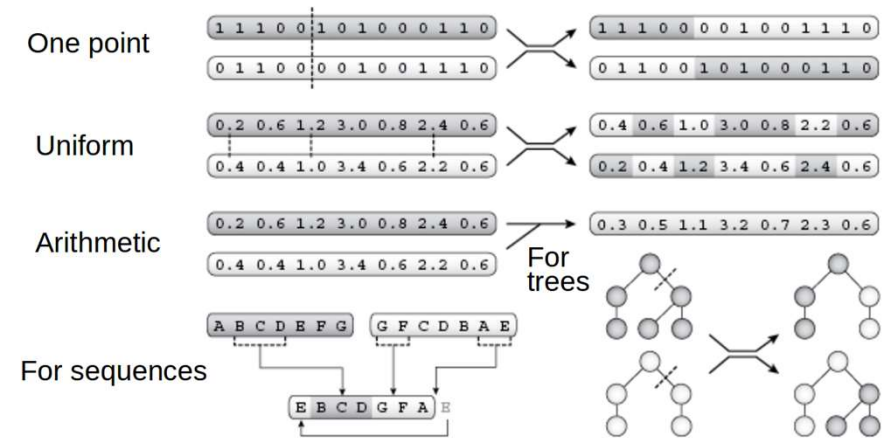
The population of the current generation needs to be replaced to form a new population of the next generation. There are different approaches.

generational replacement: old population is entirely replaced by offspring (most frequent method)

elitism: maintain *m* best individuals from previous generation to prevent loss of best individuals by effects of mutations or disadvantageous selection

generational rollover: insert offspring at the place of worst individuals

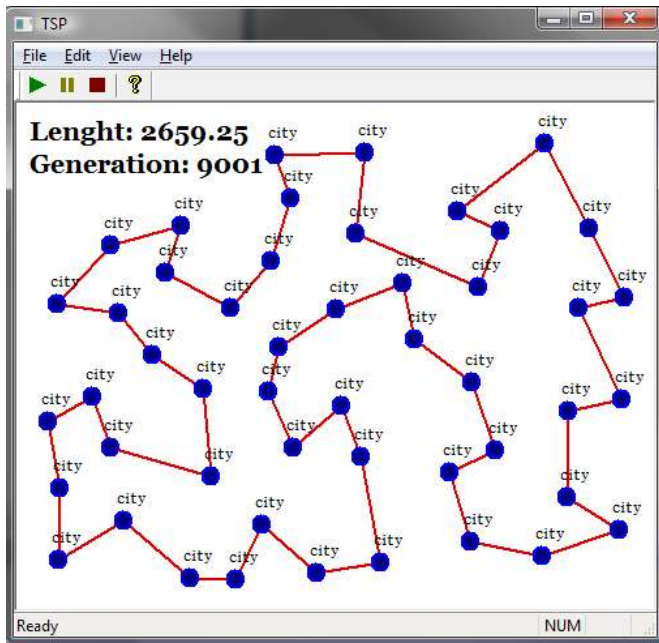
Crossover (1/2)



emulates **recombination** of genetic material from two parents during meiosis (type of cell division)

exploitation of synergy of sub-solutions (building blocks) from parents applied to randomly paired offspring with a certain probability

Crossover (2/2)



What are pre-conditions for an effective exploitation of synergies between sub-solutions via crossover?

Modularization:

sub-solutions should be independent, self-contained;
increased difficulty if the two sub-solutions depend on features encoded somewhere else in the genome

modularization is a property of the problem and the genetic representation

Example TSP:

2 good, non-overlapping sequences within permutation

parent 1: (1,3,2,4,5,6,7,8,9)

parent 2: (1,2,3,4,5,8,6,7,9)

offspring: (1,3,2,4,5,8,6,7,9)

Mutation

emulates genetic mutations
(due to radiation, errors in replication etc.)

exploration of variation of existing solutions
⇒ local search

applied to each character in the genotype
with a certain probability

Binary genotypes

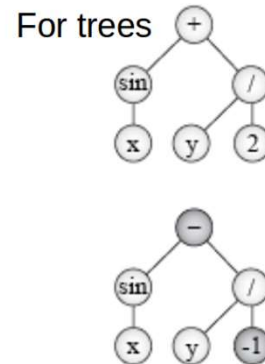
1 1 1 0 0 1 0 1 0 0 0 1 1 0
1 1 0 0 0 1 0 1 1 0 0 1 1 0

Real-valued genotypes

0.2 0.6 1.2 3.0 0.8 2.4 0.6
0.2 0.7 1.2 3.0 0.8 2.2 0.6

Sequence genotypes

G F C D B A E
G F C E B A D



Family of evolutionary algorithms

There is a whole family of evolutionary algorithm techniques. Here we have a look on only the most important ones:

Genetic algorithm (GA), Holland, 1975:

- binary genotypes, crossover, and mutation

Genetic programming (GP), Koza, 1992:

- tree-based genotypes, crossover and mutations

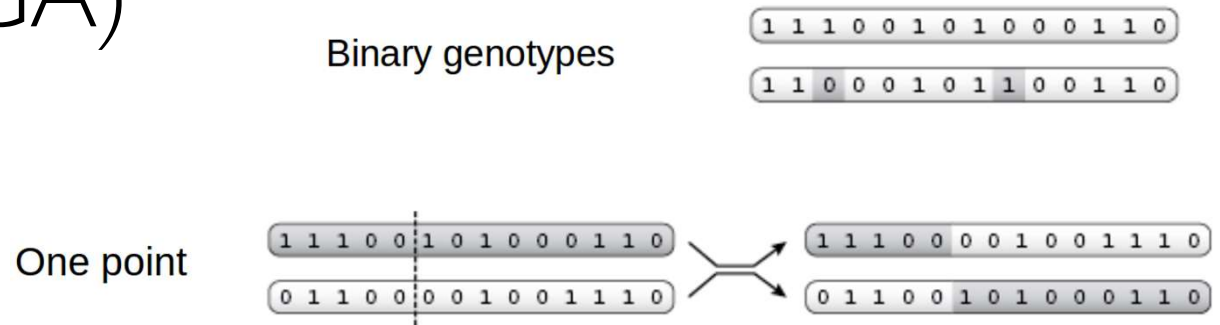
Evolution strategy (ES), Rechenberg, 1973:

- real-valued genotypes, mutations, tournaments, gradual pop. replacement, mutation range encoded in genotype of individual

Genetic algorithm (GA)

by John H. Holland, 1975

main properties:
binary genotypes, crossover, and mutation



Q: What is a good way of encoding numbers in binary code concerning mutations and local search? Answer is Gray code

Showing integers in gray code: two successive integer values differ in only one bit of their binary form (Hamming distance of 1)

Gray Code (Excurs)

Value (v)	(b=v-1) (binary)	Gray (3-bit)	Gray (decimal)
1	000	000	0
2	001	001	1
3	010	011	3
4	011	010	2
5	100	110	6

Hamming distances between successive values:

- 000 → 001 : distance 1
- 001 → 011 : distance 1

Genetic programming (GP)

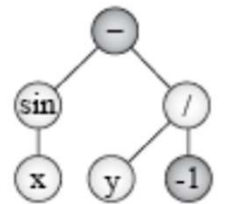
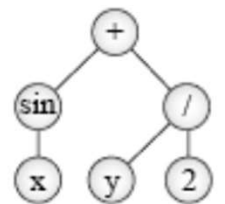
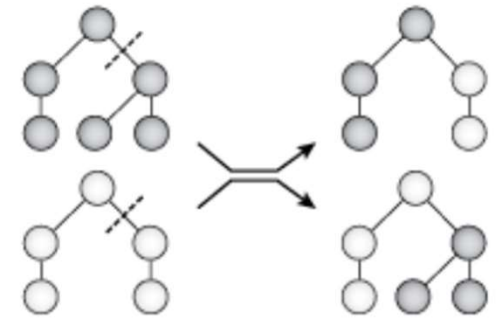
by John R. Koza, 1992

main properties:

genomes represent programs, tree-based genotypes, crossover and mutations

Q: Crossovers might be problematic – why?

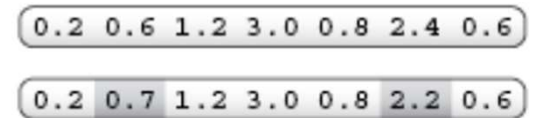
(see also Nolfi & Floreano p. 44)



Evolution strategy (ES)

by Ingo Rechenberg, 1973

Real-valued genotypes



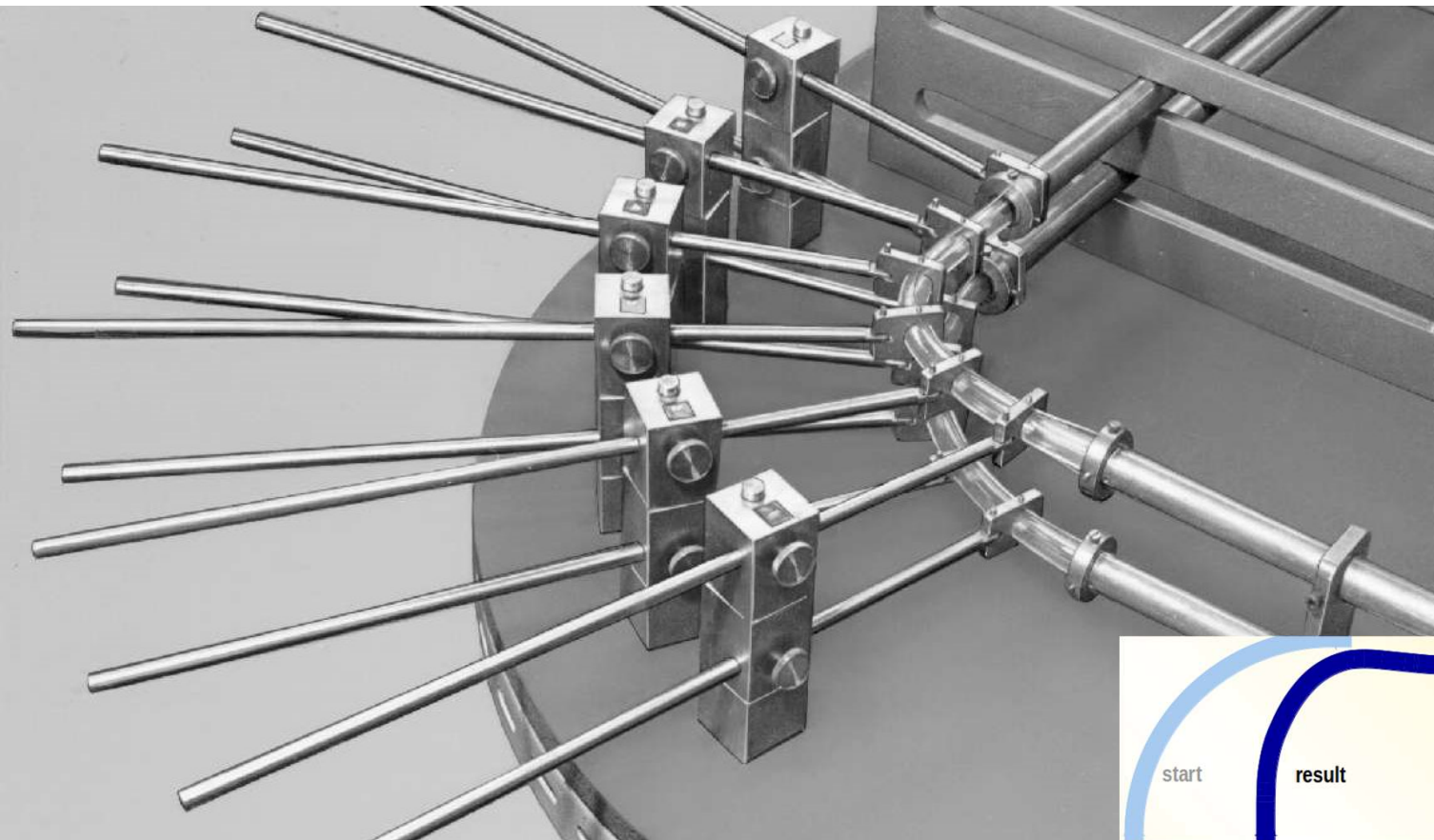
main properties:

real-valued genotypes, mutations,

mutation range encoded in genotype of individual (!)

Q: What might be disadvantages and advantages of not having a recombination operator?

Evolution strategy – early experiments (1/2)



hand-operated

